Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
```

Load Dataset

```
df = pd.read_csv('/content/Mall_Customers.csv')
df.head()
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
print(df.info())
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Genre                   200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
None
       CustomerID         Age  Annual Income (k$)  Spending Score (1-100)
count  200.000000  200.000000          200.000000              200.000000
mean   100.500000   38.850000           60.560000               50.200000
std     57.879185   13.969007           26.264721               25.823522
min      1.000000   18.000000           15.000000                1.000000
25%     50.750000   28.750000           41.500000               34.750000
50%    100.500000   36.000000           61.500000               50.000000
75%    150.250000   49.000000           78.000000               73.000000
max    200.000000   70.000000          137.000000               99.000000
```

Data Preprocessing

```
# Remove missing values
df = df.dropna()

# Convert Gender into numeric
df['Genre'] = df['Genre'].map({'Male': 0, 'Female': 1})

# Select important features
X = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
```

Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

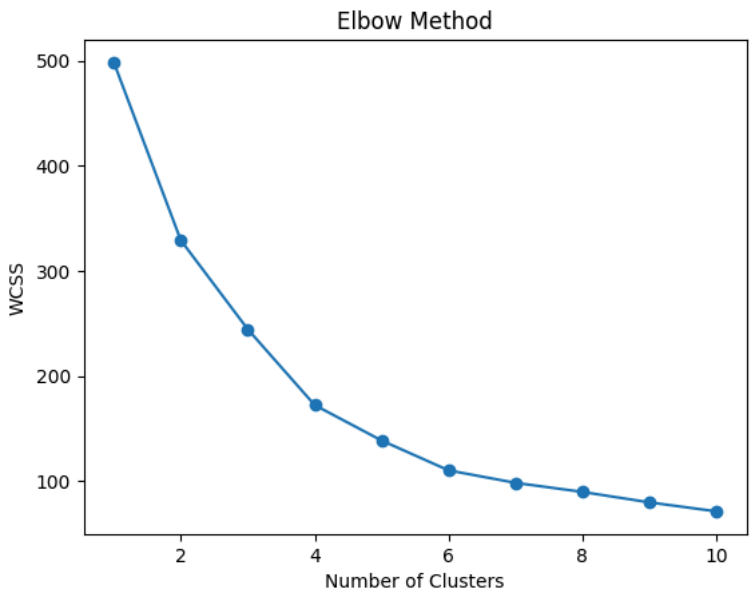
Train-Test Split

```
X_train, X_test = train_test_split(X_scaled, test_size=0.2, random_state=42)
```

Elbow Method (Find Best K)

```
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_train)
    wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```



K-Means Clustering

```
kmeans = KMeans(n_clusters=5, random_state=42)
kmeans_labels = kmeans.fit_predict(X_train)

# Evaluation
kmeans_score = silhouette_score(X_train, kmeans_labels)
print("K-Means Silhouette Score:", kmeans_score)
```

```
K-Means Silhouette Score: 0.41940649770537275
```

Hierarchical Clustering

```
hc = AgglomerativeClustering(n_clusters=5)
hc_labels = hc.fit_predict(X_train)

# Evaluation
hc_score = silhouette_score(X_train, hc_labels)
print("Hierarchical Silhouette Score:", hc_score)
```

```
Hierarchical Silhouette Score: 0.41303349446388327
```

Visualization

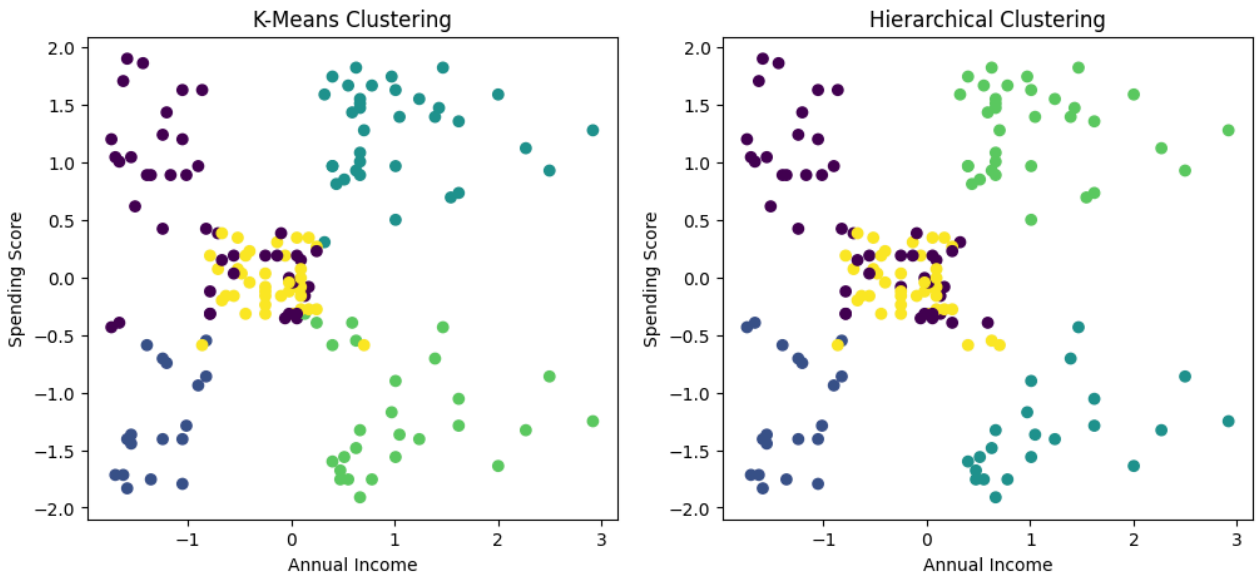
```
plt.figure(figsize=(12,5))

# K-Means
plt.subplot(1,2,1)
plt.scatter(X_train[:,1], X_train[:,2], c=kmeans_labels)
plt.title("K-Means Clustering")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")

# Hierarchical
plt.subplot(1,2,2)
plt.scatter(X_train[:,1], X_train[:,2], c=hc_labels)
plt.title("Hierarchical Clustering")
plt.xlabel("Annual Income")
```

```
plt.ylabel("Spending Score")

plt.show()
```



Find Profitable Customers

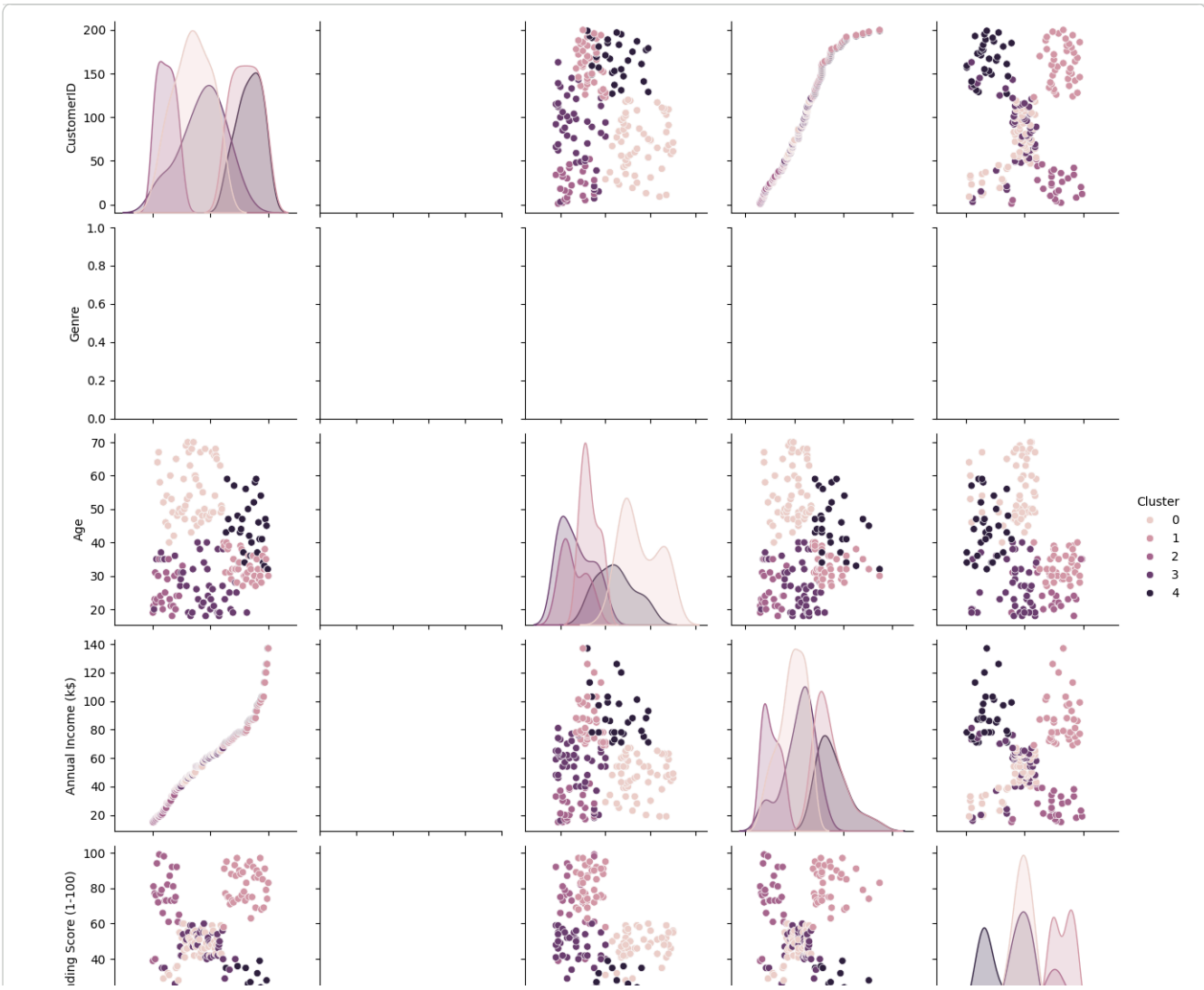
```
# Apply KMeans on full dataset
df['Cluster'] = KMeans(n_clusters=5, random_state=42).fit_predict(X_scaled)

# Group analysis
cluster_summary = df.groupby('Cluster')[['Annual Income (k$)', 'Spending Score (1-100)']].mean()
print(cluster_summary)
```

```
         Annual Income (k$)  Spending Score (1-100)
Cluster
0                 47.620690               41.706897
1                 86.100000               81.525000
2                 26.115385               74.846154
3                 54.311111               40.911111
4                 89.774194               18.483871
```

Pairplot

```
sns.pairplot(df, hue='Cluster')
plt.show()
```


CustomerID
Genre
Age
Annual Income (k$)
ding Score (1-100)
Cluster
0
1
2
3
4